



Parallel Programming Course Work 1

UP2066541

Date: 4/12/2024

Contents

Parallel Programming Course Work 1	1
Introduction.....	5
Lab Sessions	5
Lab 1: Java Threads for Parallelism.....	5
Introduction.....	5
Work	5
Results	6
Conclusion	6
Lab 2: Threads & Data	7
Introduction.....	7
Work	7
Results	10
Conclusion	11
Lab 3: Workload Decomposition & Simulation	11
Introduction.....	11
Work	11
Results	12
Conclusion	14
Lab 4: Interacting Threads	15
Introduction.....	15
Work	15
Results	16
Conclusion	16
Lab 5: Running MPJ Programs	16
Introduction.....	16
Work	16
Results	17
Conclusion	18
Lab 6: MPJ Communication	18
Introduction.....	18
Work	18
Results	19
Conclusion	20
Lab 7: Features of MPI.....	20
Introduction.....	20
Work	20
Results	20
Conclusion	21
Mini Lab: Aparapi	21
Introduction.....	21
Work	22

Results	23
Conclusion	24
Mini Project : Parallelising Ray Tracing Program	24
Introduction/Problem Statement	24
Work	24
Results	29
Conclusion	30
Conclusion	31

Table 1: Pi Outputs	6
Table 2: Average Time in Milliseconds	6
Table 3: Average Time in Nanoseconds	6
Table 4: Mandelbrot Calculation Error	7
Table 5: Parallel Speedup of Mandelbrot Approaches	11
Table 6: Generalised Mandelbrot Threads	11
Table 7: When N isn't a multiple of P	11
Table 8: Solution to Handle Cases Where N Isn't a Multiple of P	12
Table 9: Average Time & Parallel Speed Up on Generalised Mandelbrot	12
Table 10: Average Time & Parallel Speed Up on Cyclic Decomposed Mandelbrot	13
Table 11: Sequential Game of Life Ran On Different Sized Boards	13
Table 12: Generalized Parallel Mandelbrot Average Time (Ms) & Speed Up	14
Table 13: Iteration & Program Run Times for Game of Life	14
Table 14: Speed Up & Efficiency for Laplace	15
Table 15: OpenMP Code	15
Table 16: OpenMP Laplace Speedup & Efficiency	15
Table 17: Hello World MPJ Program Outputs	16
Table 18: Multicore Pi Calculation Where N is 100,000,000	17
Table 19: Cluster Mode Pi Calculation	17
Table 20: Multicore MPJ Approach to Laplace	18
Table 21: Cluster Mode MPJ Approach to Laplace	18
Table 22: DisplayPi Function Updated	19
Table 23: Gather Method Timings	19
Table 24: Replacing Redundant Ghost Regions & Increasing N in Cluster Mode	19
Table 25: MPJ Vertical Split Task Farm	20
Table 26: Square Block Split MPJ Task Farm	21
Table 27: Mini Lab Timings	23
Table 28: Prime Calculation Sequential and Parallel Code	24
Table 29: Sequential vs Parallel Prime Average Time	25
Figure 1: Sequential Pi Code	5
Figure 2: Parallel Pi Thread Code	5
Figure 3: Sequential Mandelbrot Code	7
Figure 4: Horizontal Mandelbrot Code	8
Figure 5: Vertical Implementation of Mandelbrot Set	8
Figure 6: Vertically Divided Quadcore Mandelbrot	9
Figure 7: Quadcore Mandelbrot Split Horizontally & Vertically	9
Figure 8: Timings for Different Mandelbrot Approaches (MS)	10
Figure 9: Best Times for Different Mandelbrot Approaches (MS)	10
Figure 10: Added Ghost Regions Using Barriers to Calculate Borders	12
Figure 11: Laplace Parallel vs OpenMP Average Time	16
Figure 12: Laplace Parallel vs OpenMP Speedup & Efficiency	16
Figure 13: Parallel Speedup for Multicore MPJPI	17
Figure 14: Parallel Speedup Cluster Mode MPJPI	18
Figure 15: Slow Mandelbrot VS Graphical Overhead	20
Figure 16: Sequential Matrix Multiplication Gigaflops	22
Figure 17: Sequential Matrix Multiplication Milliseconds	22
Figure 18: Improved Cache Performance Gigaflops	23
Figure 19: Improved Cache Performance Milliseconds	23
Figure 20: Gigaflops for Each Approach	24
Figure 21: Prime Parallel Speedup & Efficiency	26
Figure 22: N increased to 10,000,000 Speedup & Efficiency	26
Figure 23: MPJ Task Farm Speedup & Efficiency	27
Figure 24: Sequential Vs Sieve of Eratosthenes Best Times	27
Figure 25: Parallel SofE Speedup & Efficiency	28
Figure 26: MPJ SofE Speedup & Efficiency	28
Figure 27: Cyclic Decomposition Speedup & Efficiency	28

Figure 28: Aparapi Speedup & Efficiency for Prime Problem	29
Figure 29: Parallel Speedup of All CPU Approaches	29
Figure 30: Efficiency of All CPU Approaches	30
Figure 31: 2-N vs SofE Times	30

Introduction

This lab report aims to evaluate how parallel architectures, like distributed/shared memory, and environments are used for specific computational tasks as well as analysing computational problems to see how to exploit parallelism. This will be achieved through estimating and measuring performance using parallel speedup, gigaflops and parallel efficiency. The lab will also explore effective decomposition strategies and how synchronization methods are used to handle race conditions, such as barriers and their overhead. Additionally, the lab will investigate how task farming can be leveraged for scalability, especially with embarrassingly parallel problems. Each section aims to develop parallel solutions that improve on the original implementation.

This will be accomplished through weekly lab session experiments and a mini project investigating how a parallel solution to a prime number sieve can impact performance.

Lab Sessions

Lab 1: Java Threads for Parallelism

Introduction

For this introductory lab, a sequential program for calculating Pi was created. The experiment involved comparing a sequential and parallel approaches as well as seeing how using nano/milliseconds impacts benchmarking.

Work

Sequential Pi

Figure 1: Sequential Pi Code

```
int numSteps = 10000000; //number of subdivisions. The more divisions the higher the accuracy of approximation

double step = 1.0 / (double) numSteps; //This value represents how wide each rectangle will be. For example, if numSteps
is 10, then each rectangle will have a width of 0.1

double sum = 0.0;

for(int i = 0 ; i < numSteps ; i++){
    double x = (i + 0.5) * step ; //calculates midpoint of subdivision
    sum += 4.0 / (1.0 + x * x);
}

double pi = step * sum ;
```

Pi is calculated using a for loop which is run 10,000,000 times before returning the combined total.

Parallel Pi

Figure 2: Parallel Pi Thread Code

```

ParallelPi thread1 = new ParallelPi();
thread1.begin = 0 ; //processes the first half (from 0 to numSteps/2).
thread1.end = numSteps / 2 ;

ParallelPi thread2 = new ParallelPi();
thread2.begin = numSteps / 2 ; //processes 2nd half
thread2.end = numSteps ;
thread1.start(); // invokes the run() method in each thread concurrently.
thread2.start();
thread1.join
thread2.join();

```

To parallelise the program, two threads are created. Thread 1 handles the first half of the steps while thread 2 manages the second half. The start function makes the two threads run concurrently, whilst the join function ensures both threads have finished executing before executing the rest of the code. This is essential for ensuring correctness in the code.

Results

Table 1: Pi Outputs

	Wikipedia	Sequential	Parallel
π Value	3.141592653589793	3.141592653589731	3.141592653589923

The final digits in both programs differ to Wikipedia's given Pi value. To improve the accuracy of Pi, the number of steps should be increased until a suitable accuracy is achieved.

Table 2: Average Time in Milliseconds

Implementation	Average Time	Parallel Speed up
Sequential	32.4	-
Dual	28.4	1.14
Quad	27.6	1.17
Multi 8	52.8	0.61
Multi 16	60.2	0.53

Table 3: Average Time in Nanoseconds

Threads	Average Time	Parallel Speed up
1	24,132,633	-
2	23,200,233	1.04
4	24,450,700	0.98
8	27,482,300	0.87
16	32,360,933	0.74

Conclusion

Ensuring the threads run concurrently, is key part of parallel programming, as this is what differentiates from sequential programming and is what improves the performance.

Benchmarking in milliseconds provides an easier to interpret measure of performance as there's less digits however using nanoseconds gives greater detail, allowing for improved comparisons.

Both measures show a similar pattern of performance suggesting a dual parallel approach provides ~1.09 parallel speedup. The quad parallel approach differs slightly between timings but both Tables 2&3 show that increasing threads to

8 and 16 begins to reduce the efficiency. This result could be explained due to the machine running the program having 4 cores.

The lack of significant improvements in speed up may suggest that number of steps doesn't outweigh the cost of thread creation. Increasing N maybe provide a more accurate Pi value as well as justify a parallel approach.

Lab 2: Threads & Data

Introduction

The aim of this lab session was to investigate the speed of execution on the Mandelbrot set using multiple threads on a multicore processor. The approach to decomposing the problem for parallel execution was a key focus of the investigation. Specifically, the experiment examined how the work could be partitioned vertically, horizontally, or through a combination of both strategies.

Work

Sequential Mandelbrot

Figure 3: Sequential Mandelbrot Code

```
for(int i = 0 ; i < N ; i++) { // i and j iterate over each pixel of the image
    for(int j = 0 ; j < N ; j++) {
        double cr = (4.0 * i - 2 * N) / N ;
        double ci = (4.0 * j - 2 * N) / N ;
        double zr = cr, zi = ci ;
        int k = 0 ;
        while (k < CUTOFF && zr * zr + zi * zi < 4.0) {
            double newr = cr + zr * zr - zi * zi ;
            double newi = ci + 2 * zr * zi ;
            zr = newr ;
            zi = newi ;
            k++ ;
        }
        set [i] [j] = k ;
    }
}
```

A nested loop calculates whether values are in Mandelbrot set of size N. Cutoff is the max number of iterations to determine whether it is in the set or not.

Horizontal Mandelbrot

Table 4: Mandelbrot Calculation Error

	Before	After
Code	<pre>for(int i = begin; i< end; i++) { for(int j = begin; j < end; j++){</pre>	<pre>for(int i = begin; i< end; i++) { for (int j=0; j < N; j++) {</pre>

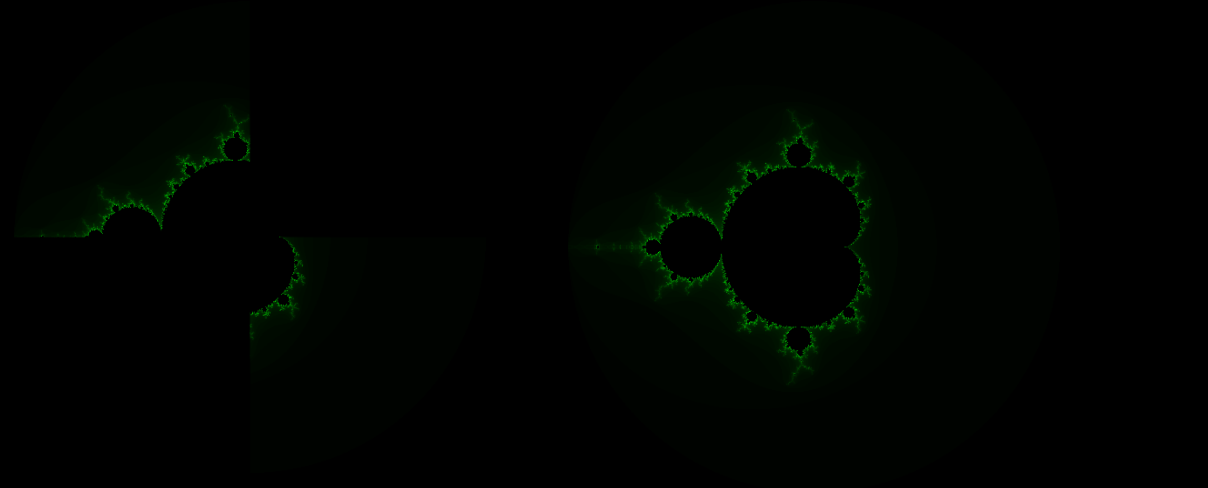
Output		
Explanation	Only the top left and bottom right are being calculated due to both 'i' and 'j' being divided.	By only using the beginning and end on the first loop, all of the Mandelbrot set is calculated.

Figure 4: Horizontal Mandelbrot Code

```
ParallelMandelbrot thread0 = new ParallelMandelbrot(0);
ParallelMandelbrot thread1 = new ParallelMandelbrot(1);

thread0.start();
thread1.start();

thread0.join();
thread1.join();
public ParallelMandelbrot(int me) {
    this.me = me; // used to determine which thread is being executed
}
public void run() {
    int begin, end;
    //thread 0
    if (me == 0) {
        begin = 0;
        end = N/2;
    }
    else { // me == 1
        begin = N/2;
        end = N;
    }
    for(int i = begin; i < end; i++) {
        for (int j=0; j < N; j++) {
            //calculate Mandelbrot Here
        }
    }
}
```

In this approach the problem is decomposed blockwise so that two threads can simultaneously calculate half of the image, which theoretically should half the run time as the number of steps needed in the sequential approach is halved between them. The main difference in the way the Mandelbrot and pi threads are created is that the pi threads begin and end points are specified before starting the run function, while the Mandelbrot threads use 'this' to refer to the specific thread object with a given number which is used in the run function to determine which steps a thread should perform.

Vertical Mandelbrot

Figure 5: Vertical Implementation of Mandelbrot Set

```
for(int i = 0; i < N; i++) {
    for (int j = begin; j < end; j++) {
```

To divide the steps vertically, swap 'i' and 'j's begin and end values.

Quadcore Mandelbrot

Figure 6: Vertically Divided Quadcore Mandelbrot

```
QuadcoreMandelbrot thread0 = new QuadcoreMandelbrot(0);
QuadcoreMandelbrot thread1 = new QuadcoreMandelbrot(1);
QuadcoreMandelbrot thread2 = new QuadcoreMandelbrot(2);
QuadcoreMandelbrot thread3 = new QuadcoreMandelbrot(3);
thread0.start();
thread1.start();
thread2.start();
thread3.start();
thread0.join();
thread1.join();
thread2.join();
thread3.join();
...
//run function
if (me == 0) { //thread 0
    begin = 0;
    end = N/4;
}
else if (me == 1) { //thread 1
    begin = N/4;
    end = N/2;
}
else if (me == 2) { //thread 2
    begin = N/2;
    end = (N/4) * 3;
}
else { // me == 3
    begin = (N/4) * 3;
    end = N;
}
for (int i = 0; i < N; i++) {
    for (int j = begin; j < end; j++) {
```

Adding two more threads will partition the workload between four threads which can run concurrently.

Figure 7: Quadcore Mandelbrot Split Horizontally & Vertically

```
int beginX, endX, beginY, endY;

if (me == 0) { // Top-left quadrant
    beginX = 0;
    endX = N / 2;
    beginY = 0;
    endY = N / 2;
} else if (me == 1) {
    beginX = N / 2;
    endX = N;
    beginY = 0;
    endY = N / 2;
} else if (me == 2) {
    beginX = 0;
    endX = N / 2;
    beginY = N / 2;
    endY = N;
} else { // Bottom-right quadrant
    beginX = N / 2;
    endX = N;
    beginY = N / 2;
    endY = N;
```

```

}
for (int i = beginY; i < endY; i++) {
    for (int j = beginX; j < endX; j++) {...

```

The partitions can also be split both horizontally and vertically.

Results

Figure 8: Timings for Different Mandelbrot Approaches (MS)

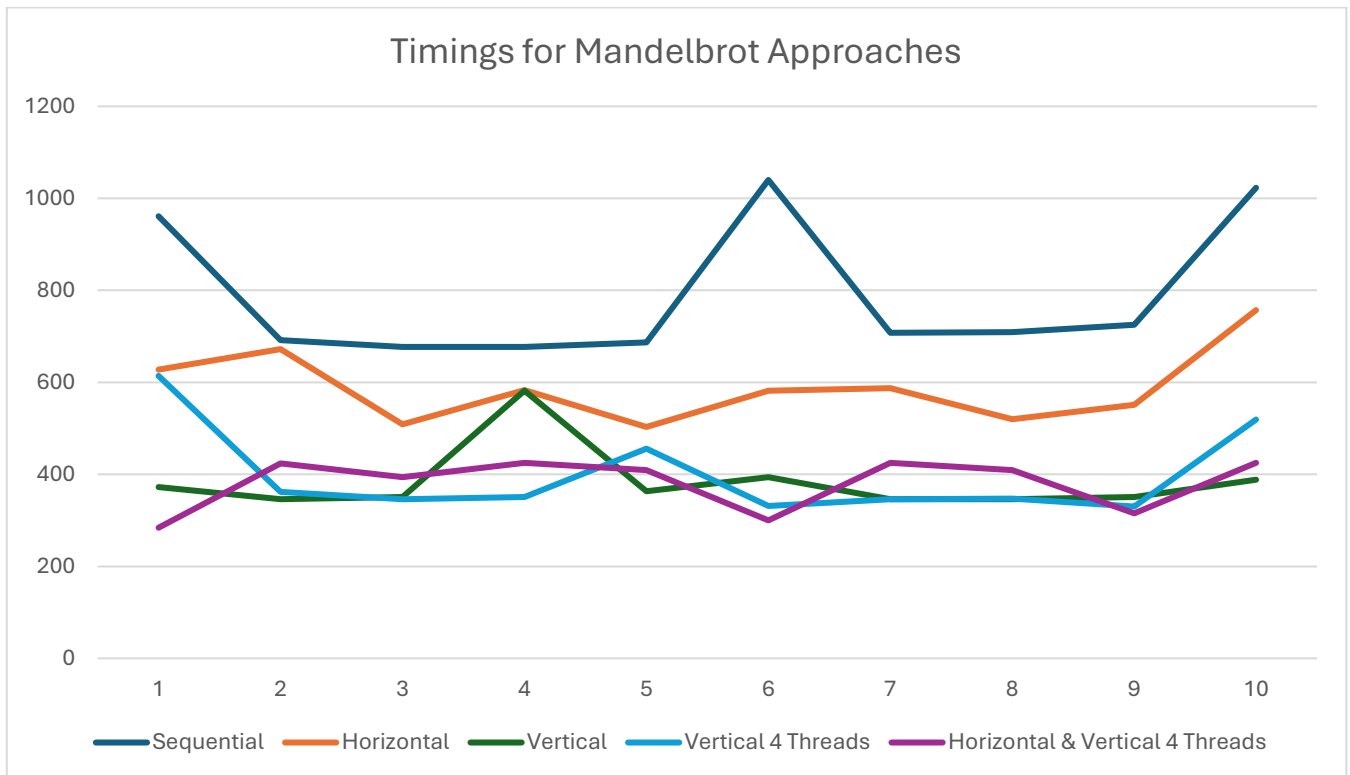


Figure 9: Best Times for Different Mandelbrot Approaches (MS)

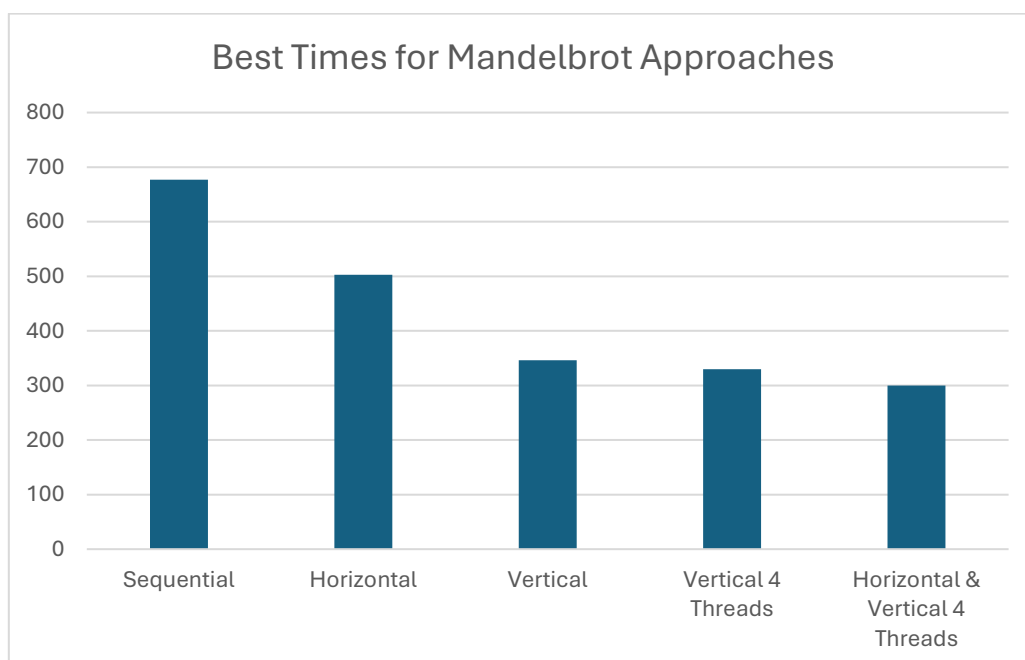


Table 5: Parallel Speedup of Mandelbrot Approaches

Approach	Parallel Speedup
Horizontal 2 Thread	1.34
Vertical 2 Thread	1.95
Vertical 4 Threads	2.05
Horizontal & Vertical 4 Threads	2.25

The horizontal and vertical four thread approach had the fastest run time, as well as the greatest parallel speed up. It also had consistent timings unlike the sequential approach.

Conclusion

How the workload is divided across threads can impact the performance of a parallel program. As seen in Table 5, despite both using two threads, the vertical approach outperforms the horizontal one. This is due to it having better workload balancing.

Lab 3: Workload Decomposition & Simulation

Introduction

In this lab session, the aim was to generalize the parallel Mandelbrot program to run on a given number of threads, as well as create a sequential and parallel Game of Life simulation, exploring barrier synchronisation.

Work

Generalising Mandelbrot Thread Creation

Table 6: Generalised Mandelbrot Threads

Before	After
<pre>ParallelMandelbrot thread0 = new ParallelMandelbrot(0) ; ParallelMandelbrot thread1 = new ParallelMandelbrot(1) ; thread0.start() ; thread1.start() ; thread0.join() ; thread1.join() ;</pre>	<pre>final static int P = 2; ... ParallelMandelbrot [] threads = new ParallelMandelbrot [P] ; for(int me = 0 ; me < P ; me++) { threads [me] = new ParallelMandelbrot(me) ; threads [me].start() ; } for(int me = 0 ; me < P ; me++) { threads [me].join() ; }</pre>
Originally each thread object is assigned to a variable where the start function executes the threads, and the join ensure that all threads are finished executing before moving on. This code is suitable for the creation of a few threads, however as the number of threads increases so will the code.	By generalising the code using the constant P, a list of threads can be created using loops resulting in the same number of lines despite having a varying number of threads. Starting and joining need to be kept in separate loops as parallelism can't be achieved if all the threads haven't been created before waiting for their execution.

Table 7: When N isn't a multiple of P

N	P	B
4096	2	2048
4095	2	2047

Dividing 4095 by 2 returns 2047.5, however b is an integer value of 2047. In this scenario where N is not a multiple of P, there will be remaining pixels not included in the calculations. And as a pixel can't be split in half, the best solution is to assign each remainder to a thread. This will result in an unbalanced workload however it will ensure every pixel is calculated.

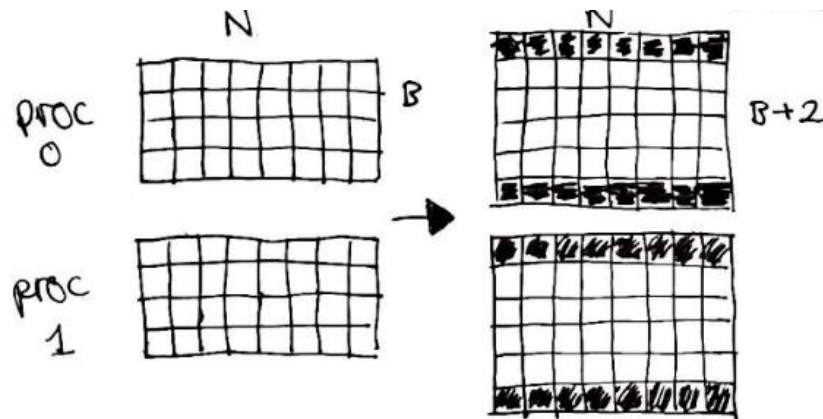
Table 8: Solution to Handle Cases Where N Isn't a Multiple of P

Before	After
<pre>int begin, end ; int b = N/P ; // block size System.out.println(b); begin = me * b ; end = begin + b ;</pre>	<pre>int begin, end ; int b = N/P ; int remainder = N % P; if (N % P == 0) { begin = me * b; end = begin + b; } else { if (me < remainder) { begin = me * (b + 1); end = begin + (b + 1); } else { begin = me * b + remainder; end = begin + b; } }</pre>

Parallel Game of Life

Following the same method as Lab 2, but now incorporating a generalised approach for thread creation, The Game of Life was parallelised across two threads. As The Game of Life relies on neighbouring cells in order to determine whether a cell is alive or not, threads need to share bordering cell values. This can be achieved by implementing barriers and awaiting thread completion to synchronise threads.

Figure 10: Added Ghost Regions Using Barriers to Calculate Borders



Results

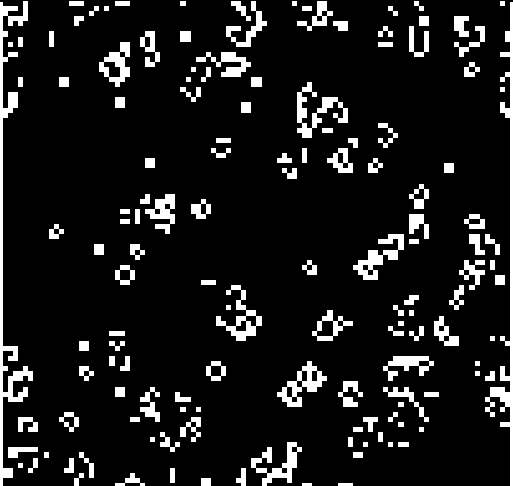
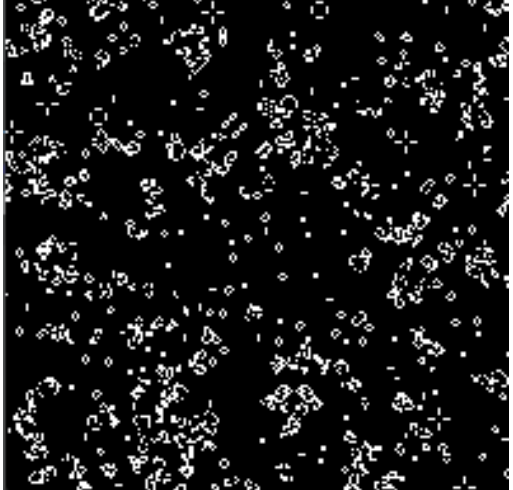
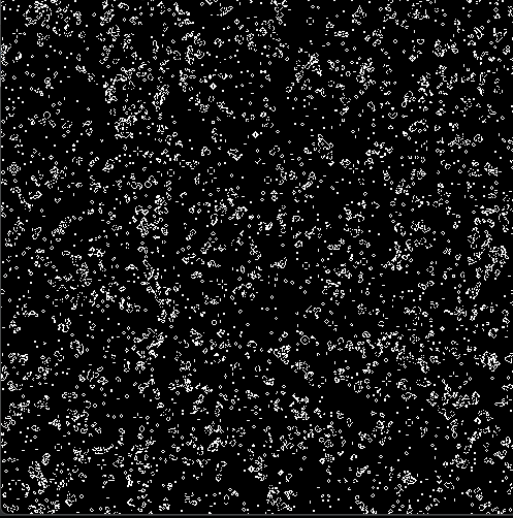
Table 9: Average Time & Parallel Speed Up on Generalised Mandelbrot

Threads	Average Time (Ms)	Parallel Speedup
2	622	1.08
4	522	1.29
8	386	1.75

Table 10: Average Time & Parallel Speed Up on Cyclic Decomposed Mandelbrot

Threads	Average Time (Ms)	Parallel Speedup
2	371	1.82
4	202	3.35
8	185	3.65

Table 11: Sequential Game of Life Ran On Different Sized Boards

Board Size	Board
100	
200	
500	

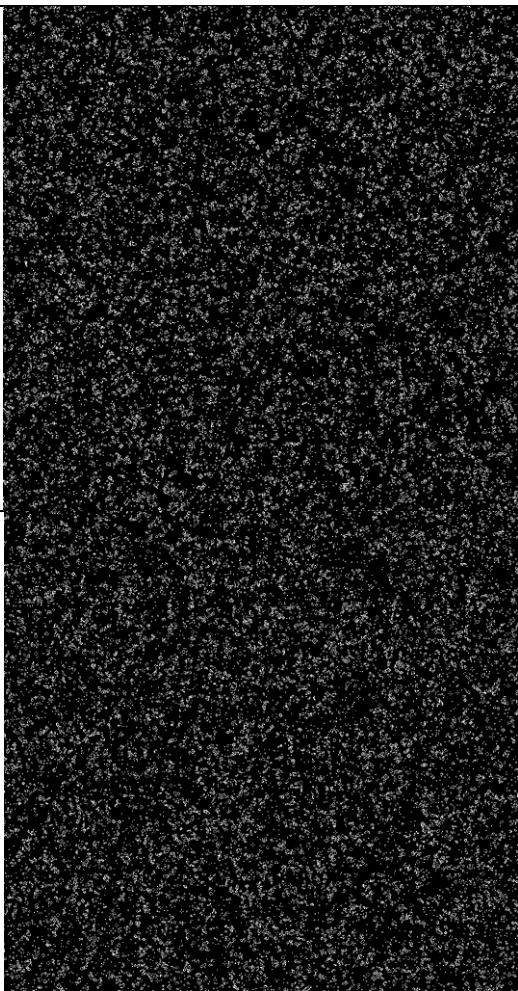
1024			
2000			

Table 12: Generalized Parallel Mandelbrot Average Time (Ms) & Speed Up

Threads		2	4	8	16
Blockwise Decompostion	Time (ms)	544	524	340	204
	Parallel Speed up	1.24	1.29	1.99	3.34
Cyclic Decompostion	Time (ms)	360	252	185	162
	Parallel Speed up	1.88	2.68	3.65	4.17

Table 13: Iteration & Program Run Times for Game of Life

Approach	Average Iteration Time (Ms)	Average Program Time (Ms)
Sequential	31.9	4824
2 Thread	31.6	2731
4 Thread	28.4	1921
8 Thread	22	1789

Conclusion

When decomposing the Mandelbrot set a cyclic approach is more efficient than blockwise. Tables 9 & 10 show that the cyclic approach was twice as fast as the generalised blockwise approach.

Lab 4: Interacting Threads

Introduction

Work

Parallel Laplace

Before	After
<pre>for(int i = 1 ; i < N - 1 ; i++) { for(int j = 1 ; j < N - 1 ; j++) { newPhi [i] [j] = 0.25F * (phi [i] [j - 1] + phi [i] [j + 1] + phi [i - 1] [j] + phi [i + 1] [j]) ; } } // Update all phi values for(int i = 1 ; i < N - 1 ; i++) { for(int j = 1 ; j < N - 1 ; j++) { phi [i] [j] = newPhi [i] [j] ; } }</pre>	<pre>// Calculate new phi for(int i = begin ; i < end ; i++) { for(int j = 1 ; j < N - 1 ; j++) { newPhi [i] [j] = 0.25F * (phi [i] [j - 1] + phi [i] [j + 1] + phi [i - 1] [j] + phi [i + 1] [j]) ; } } synch(); // Update all phi values for(int i = begin ; i < end ; i++) { for(int j = 1 ; j < N - 1 ; j++) { phi [i] [j] = newPhi [i] [j] ; } }</pre>

Table 14: Speed Up & Efficiency for Laplace

Approach	Best Time (Ms)	Parallel Speed up	Parallel Efficiency	Overhead of Barrier Calls (Ms)
Sequential	18,279	-	-	-
2 Threads	11,938	1.53	76%	990
4 Threads	11,666	1.56	36%	3,114
8 Threads	13,614	1.34	16%	9,388

OpenMP

Table 15: OpenMP Code

<pre>#pragma omp parallel for for(int i = 0 ; i < N ; i++) { for(int j = 0 ; j < N ; j++) {</pre>
The pragma omp line is called before calculating the sum and updating the board. This line is responsible for dividing the outer loop into multiple threads.

Table 16: OpenMP Laplace Speedup & Efficiency

Approach	Best Time (Ms)	Parallel Speed up	Parallel Efficiency
2 Threads	6,796	1.61	80%
4 Threads	5,397	2.03	50%
8 Threads	2,929	3.74	46%

Results

Figure 11: Laplace Parallel vs OpenMP Average Time

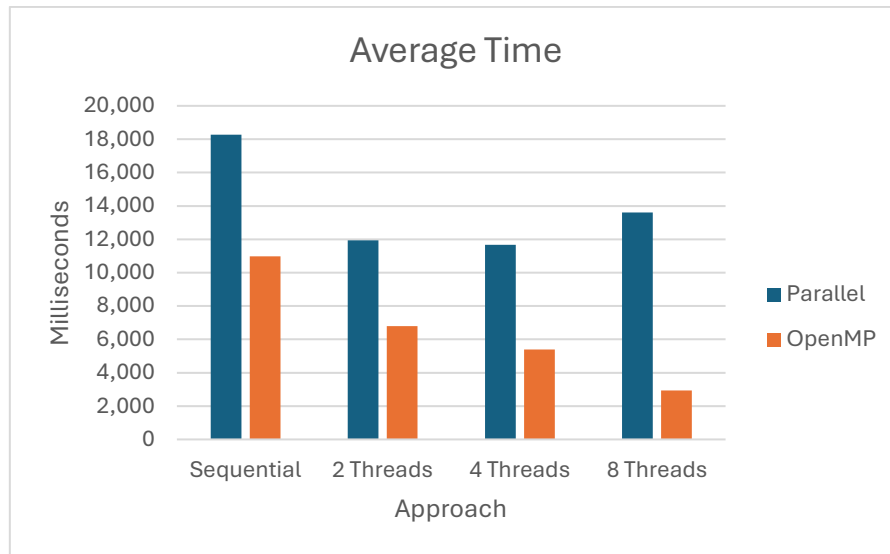
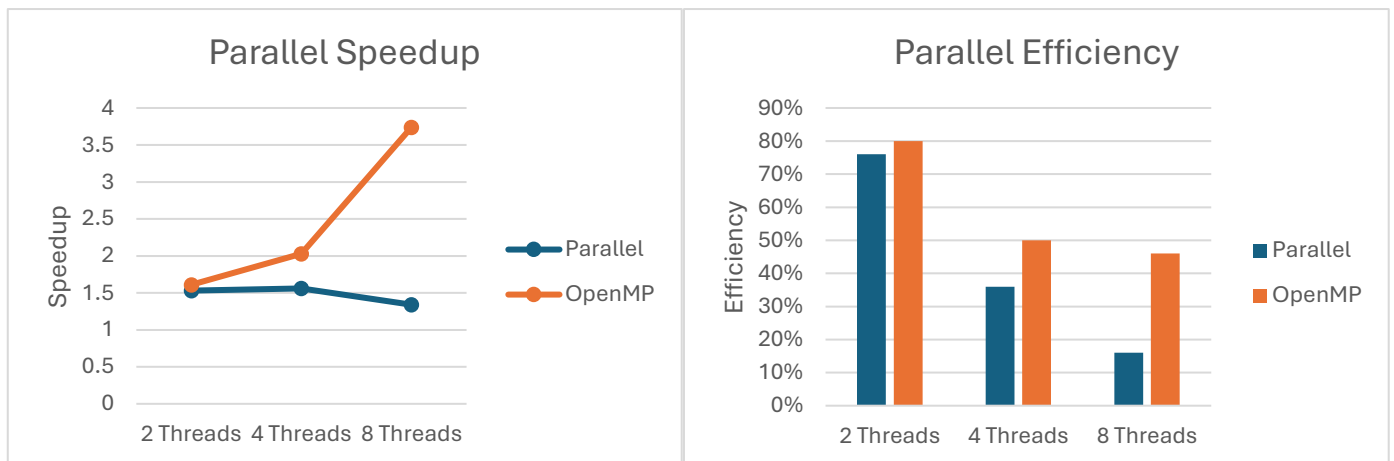


Figure 12: Laplace Parallel vs OpenMP Speedup & Efficiency



Conclusion

The optimal solution to the Laplace problem is using the OpenMP approach on 8 threads. This is due to OpenMP being optimised to run on shared memory multicore systems, as was the case in this scenario. As OpenMP automates thread management, higher performance levels can be achieved with ease unlike the Java parallel approach which requires manual fine tuning.

Barrier synchronization is necessary for the Laplace program as threads must finish executing before the next step is executed. However, this comes at a cost as the barrier calls take time.

Lab 5: Running MPJ Programs

Introduction

Work

Hello World

Table 17: Hello World MPJ Program Outputs

Number of Threads	Output
-------------------	--------

1	Hi from <0>
2	Hi from <1> Hi from <0>
4	Hi from <0> Hi from <2> Hi from <1> Hi from <3>

Approach	Average Time
Sequential	757

Results

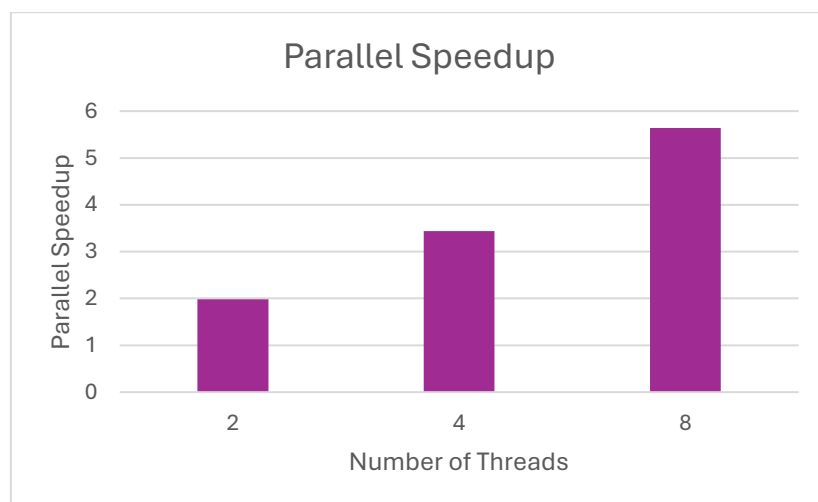
Multicore Pi Calculation

To see substantial parallel speed ups, the number of steps was increased to a billion.

Table 18: Multicore Pi Calculation Where N is 100,000,000

Processes	Average Time (ms)
2	382
4	220
8	134

Figure 13: Parallel Speedup for Multicore MPJPi



Using a multicore approach created significant parallel speedup, which increased with the number of processes.

Cluster Mode

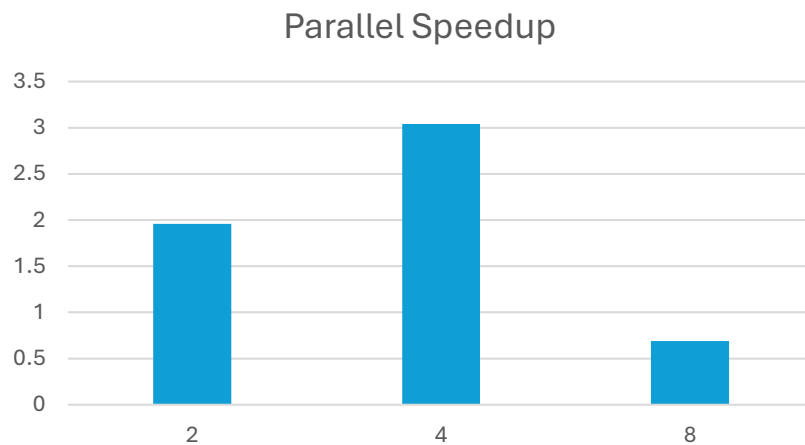
Three worker nodes ran daemon and the program was executed on the master node. This means that the task could be distributed across three nodes.

Table 19: Cluster Mode Pi Calculation

Processes	Average Time (ms)
-----------	-------------------

2	386
4	249
8	1089

Figure 14: Parallel Speedup Cluster Mode MPJPI



Conclusion

Using the MPJ approach to exploit distributed memory, allowed for greater parallel speedup compared to the shared memory approach (Table 2). The MPJ approach had slower times due to an increase in the number of steps which displayed the difference in speedups more effectively.

Theoretically the cluster mode approach should be faster than the multicore approach, however the results gathered found otherwise. This is likely due to a load imbalance and simplicity of the problem. Having an even number of workers nodes means that the processes would have been split evenly between them. Calculating a more complex problem would be more suitable for the cluster mode as this would make up for the overhead of having to distribute the work and gathering the results across separate machines.

Lab 6: MPJ Communication

Introduction

Discovering how data may be distributed across the cooperating processes of a distributed memory parallel program, is the key goal of this lab session, as well as looking at MPI communication within the Game Of Life program.

Work

Adding Ghost Regions to Laplace Solution

Table 20: Multicore MPJ Approach to Laplace

Processes	Average Time	Parallel Speedup
2	7,760	2.35
4	8,510	2.14
8	14,856	1.23
12	41,115	0.44

Running in Cluster Mode

Table 21: Cluster Mode MPJ Approach to Laplace

Processes	Best Time	Parallel Speedup
-----------	-----------	------------------

2	44,947	0.4
4	41,575	0.46
8	42,080	0.43

The approximate number of communications in the program can be estimated by taking the two send/receive calls in the edge swapping code and multiplying by the number of iterations (100,000) and then multiplying by the number of processes. To display the Laplace all processes apart from process 0 must send their calculated phi value and it must be received.

$$C = (2 \times 100,000) \times ((P - 1) \times 2)$$

Using Gather Method

Table 22: DisplayPi Function Updated

<pre> public static void displayPhi() { if(me > 0) { MPI.COMM_WORLD.Send(phi, 1, B, MPI.OBJECT, 0, 0); } else { // me == 0 for(int i = 1; i <= B; i++) { for(int j = 0; j < N; j++) { allPhi[i - 1][j] = phi[i][j]; } } for(int src = 1; src < P; src++) { MPI.COMM_WORLD.Recv(allPhi, src * B, B, MPI.OBJECT, src, 0); } display.repaint(); } } </pre>	<pre> public static void displayPhi() { float[] flatPhi = new float[B * N]; for (int i = 1; i <= B; i++) { System.arraycopy(phi[i], 0, flatPhi, (i - 1) * N, N); } float[] allFlatPhi = new float[P * B * N]; MPI.COMM_WORLD.Gather(flatPhi, 0, B * N, MPI.FLOAT, allFlatPhi, 0, B * N, MPI.FLOAT, 0); if (me == 0) { 1D array allFlatPhi for (int i = 0; i < N; i++) { for (int j = 0; j < N; j++) { allPhi[i][j] = allFlatPhi[i * N + j]; } } display.repaint(); } } </pre>
In the original code, send and receive functions are used. The time complexity of these actions is based on the number of processes multiplied by the size of the message making it less efficient.	The gather function consolidates data from multiple sources into one unlike the send/ receive which transmits/retrieves from one process. Theoretically this would make the gather function faster.

Table 23: Gather Method Timings

Processes	Milliseconds
2	281,536
4	1,524,032
8	2,174,196

Results

By replacing the redundant ghost region code, theoretically this should halve the total communication making it faster, however the opposite was found. When running on 2 processes the program was 3503 milliseconds slower. This could be due to the modified approach causing a situation where one process has to wait for the other to finish sending/receiving before it can proceed, introducing more synchronization overhead.

Table 24: Replacing Redundant Ghost Regions & Increasing N in Cluster Mode

Processes	N	Average Time (MS)	Parallel Speedup
2	512	48,479	0.37
4	512	42,544	0.42
2	1024	48,327	0.37
4	1024	42,671	0.42

25	1024	39,046	1.24
----	------	--------	------

Increasing the problem size from 512 to 1024 made no difference to the speed up. In theory replacing the ghost region code and increasing the workload should speed the program up as the communication overhead will become less. Load imbalance shouldn't be an issue as four nodes were used for this task.

Conclusion

Theoretically, cluster mode should be more effective in handling the Laplace problem, especially when the workload increases, however the lab work found the opposite. Using multicore mode parallel speedups of 2.35 can be reached whilst for cluster mode the parallel speed up shows it is much slower.

By using the gather method, ideally the program should P times faster as the one send or receive per process is reduced to one for all. However, the results in table 23 show the opposite.

The results of these experiments do not align with the expected performance which could be due to incorrect implementation or the lack of computational complexity to highlight the benefits.

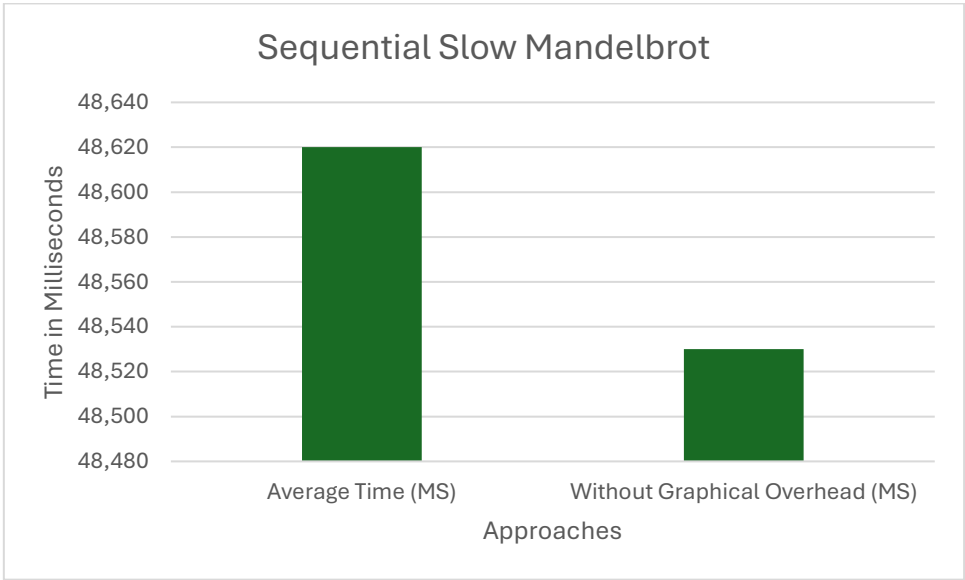
Lab 7: Features of MPI

Introduction

In this lab session task farming was introduced to dynamically approach embarrassingly parallel problems by implementing a master and worker nodes.

Work

Figure 15: Slow Mandelbrot VS Graphical Overhead



Having a high Cutoff value increases computation resulting in the sequential program running slowly. Reducing the graphical overhead by printing the display less made the program 90 milliseconds quicker. Whilst this is an improvement further steps are necessary to make a meaningful difference.

Results

Table 25: MPJ Vertical Split Task Farm

Processes	Time	Parallel Speedup
Multicore		
1	28,226	1.72

2	14,550	3.34
3	9,719	5
7	4,475	10.86
8	4,532	10.72
Cluster		
1	26,668	1.82
2	13,624	3.56
3	9,287	5.23
7	4,339	11.2
8	3,885	12.5
17	1,957	24.8
25	1,584	30.69
35	1,186	40.99

Introducing a vertical split task farm significantly improved the performance of the program. Running 7 processes provided the highest speed up when using a multicore approach. The cluster mode approach had even more of a positive impact on performance as running 35 processes had a parallel speed up of 41 times faster than the original sequential approach. Theoretically the number of processors could be pushed to 108 (with 1 being the master) as there are nine nodes with twelve cores available however the communication overhead was too great to run any higher than 35 without timing out.

Table 26: Square Block Split MPJ Task Farm

Processes	Time	Parallel Speedup
2	52,658	0.92
3	46,473	1.04
7	35,754	1.35

Splitting the workload into squares rather than vertically resulted in the program running slower than the sequential approach.

Conclusion

The lab demonstrated that task farming significantly enhances the efficiency of solving embarrassingly parallel problems. The results clearly show that task farming is highly effective for problems where tasks can be executed independently, leading to improved scalability and parallelization.

A key finding from the lab is that workload decomposition has a substantial impact on overall performance. Changing the block size had little impact on the speed of the program.

Mini Lab: Aparapi

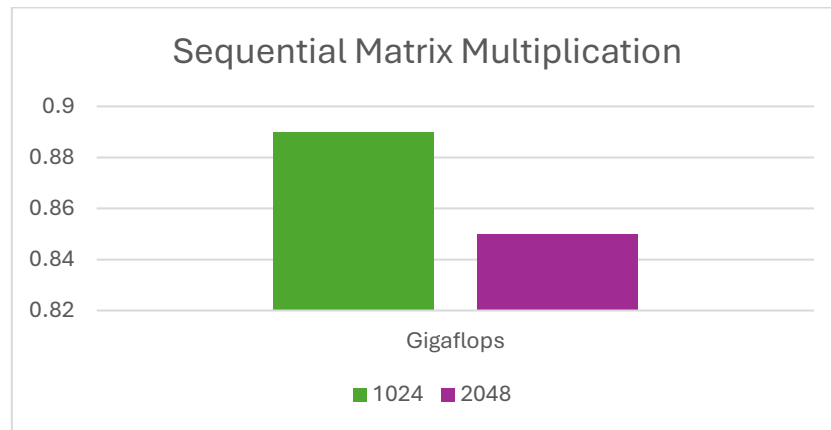
Introduction

Aparapi is introduced so parallel code can be ran on GPU's. Experiments into the performance of parallel code using Aparapi were conducted to investigate the benefits of GPU use.

Work

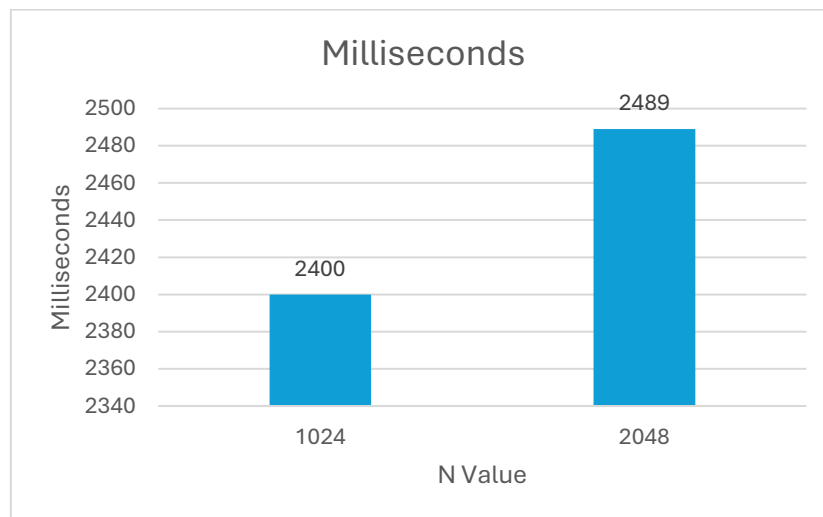
Sequential Matrix Multiplication

Figure 16: Sequential Matrix Multiplication Gigaflops



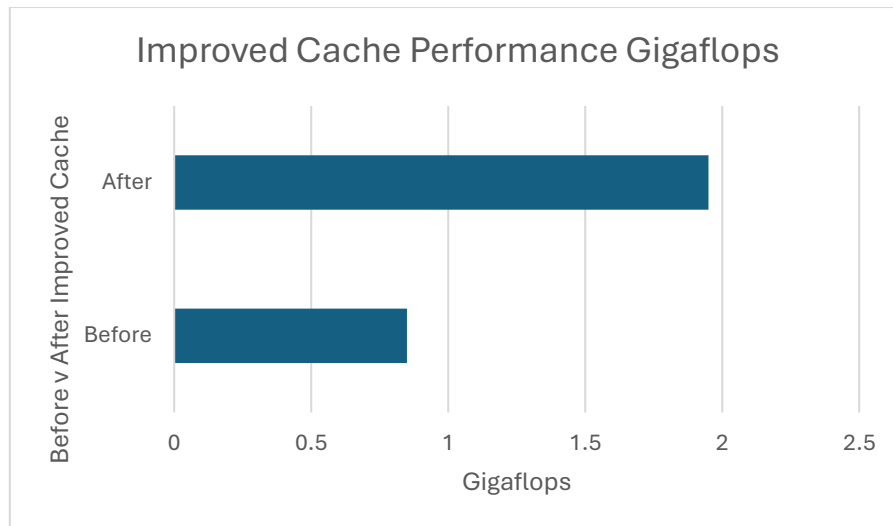
Running the program sequentially with N value of 1024 provides a modest result however as the complexity is increased, the number of gigaflops decreases indicating that there's poor scalability for computational complexity.

Figure 17: Sequential Matrix Multiplication Milliseconds



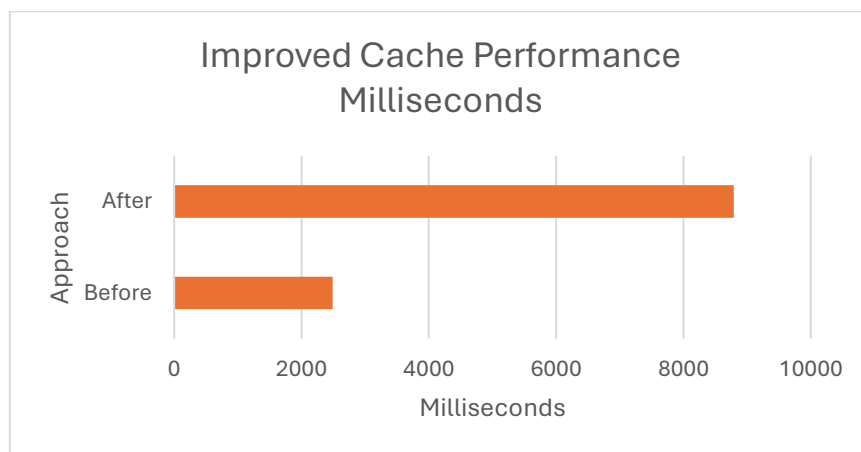
Not only does the number of gigaflops crease but the run time also increases with the N value. This further shows that the sequential approach is not suitable for larger, difficult problems.

Figure 18: Improved Cache Performance Gigaflops



Introducing a temporary variable for the transpose of b significantly improved the results obtained. By updating the loop that calculates c, data structures can now fit within the cache memory causing the number of gigaflops to double.

Figure 19: Improved Cache Performance Milliseconds



Despite the significant improvement in gigaflops, the program now runs almost four times slower than it did originally. This suggests that optimising the cache introduces overheads, which when running the program on this level of complexity, the cost outweighs the benefit from a speed perspective.

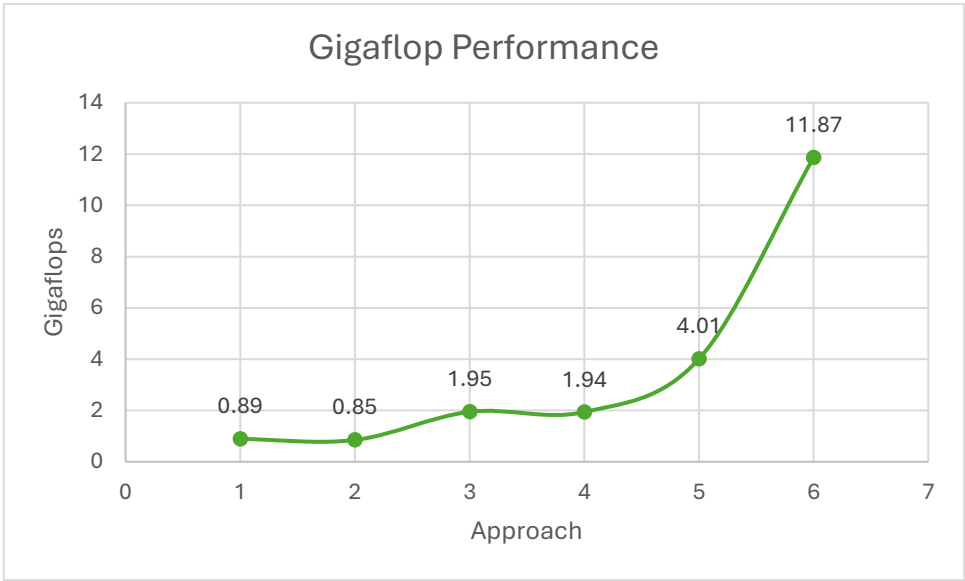
Results

Table 27: Mini Lab Timings

Approach Number	Approach	Average Time (MS)
CPU		
1	N = 1024	2,400
2	Increase N =2048	2,489
3	Improved Cache	8,789
4	Increased N = 4096	70,647
GPU		
5	Use Aparapi	4,130
6	Increase N =4096	12,171

Using aparapi doubles the speed of the program when the cache has been improved and N= is set to 2048. However, this approach is still slower than the original sequential CPU code, this is likely due to the overhead of transferring data to the GPU.

Figure 20: Gigaflops for Each Approach



Whilst the timings can vary depending on the approach, Figure 18 clearly demonstrates that using Aparapi with an optimised cache results in improved gigaflop performance. This also demonstrates the scalability of this approach and how it handles larger problems.

Conclusion

The increased number of gigaflops demonstrates that utilising GPU’s through Aparapi is effective in improving the programs performance. It should also be noted that the approach taken to code a solution can have significant impacts on performance as the original solution provided data structures too large for the cache memory, showing the importance of cache performance of CPU’s.

Mini Project : Parallelising Ray Tracing Program

Introduction/Problem Statement

The aim of this mini project is to explore how parallel programming techniques and architectures can be used to improve the performance of a simple prime number sieve. This project’s goal is also explore how different algorithmic approaches to finding all prime to a given number impacts performance and its suitability to be parallelised.

Work

Parallelisation

Table 28: Prime Calculation Sequential and Parallel Code

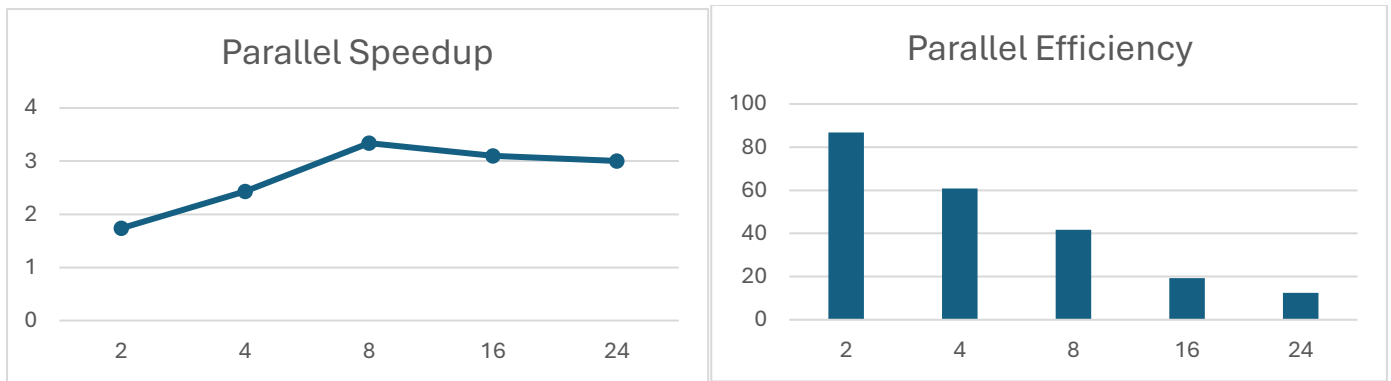
Sequential Code	Parallel Code
<pre>import java.util.ArrayList; import java.util.List; public class SequentialPrime { public static void main(String[] args) { long startTime = System.currentTimeMillis(); int N = 100000;</pre>	<pre>public class ParallelPrime extends Thread { final static int N = 100000; final static int P = 2; static List<Integer> primeList = new ArrayList<>(); public static void main(String[] args) throws Exception {</pre>

<pre> List<Integer> primeList = new ArrayList<>(); for (int n = 2; n < N; n++) { boolean isPrime = true; for (int t = 2; t <= Math.sqrt(n); t++) { if (n % t == 0) { isPrime = false; break; } } if (isPrime) { primeList.add(n); } } long endTime = System.currentTimeMillis(); System.out.println("Number of prime numbers found: "+ primeList.size()); System.out.println("Calculated in " + (endTime - startTime) + " milliseconds"); } } </pre>	<pre> ParallelPrime[] threads = new ParallelPrime[P]; long startTime = System.currentTimeMillis(); for (int me = 0; me < P; me++) { threads[me] = new ParallelPrime(me); threads[me].start(); } for (int me = 0; me < P; me++) { threads[me].join(); } long endTime = System.currentTimeMillis(); System.out.println("Number of prime numbers found: "+ primeList.size()); System.out.println("Total computation time: " + (endTime - startTime) + " milliseconds"); } int me; ParallelPrime(int me) { this.me = me; } final static int B = N / P; public void run() { int begin = (me * B) + 2 int end = begin + B; for (int n = begin; n < end; n++) { boolean isPrime = true; for (int t = 2; t <= Math.sqrt(n); t++) { if (n % t == 0) { isPrime = false; break; } } if (isPrime) { primeList.add(n); } } } } </pre>
N is set to a large number to increase computational complexity which should help display performance differences between a sequential and parallel approach. The value n is set to 2 because 1 is not a prime number. It then checks every number from 2 to 100,000 and prints if it is prime.	The prime code can be parallelised similarly to the Pi code (Figure 2). In this version the numbers to check are split between threads. Setting P to equal 2 in this code should theoretically half the time as half the calculations are running concurrently.

Table 29: Sequential vs Parallel Prime Average Time

Approach	Time in Milliseconds
Sequential	474
2 Thread Parallel	352

Figure 21: Prime Parallel Speedup & Efficiency

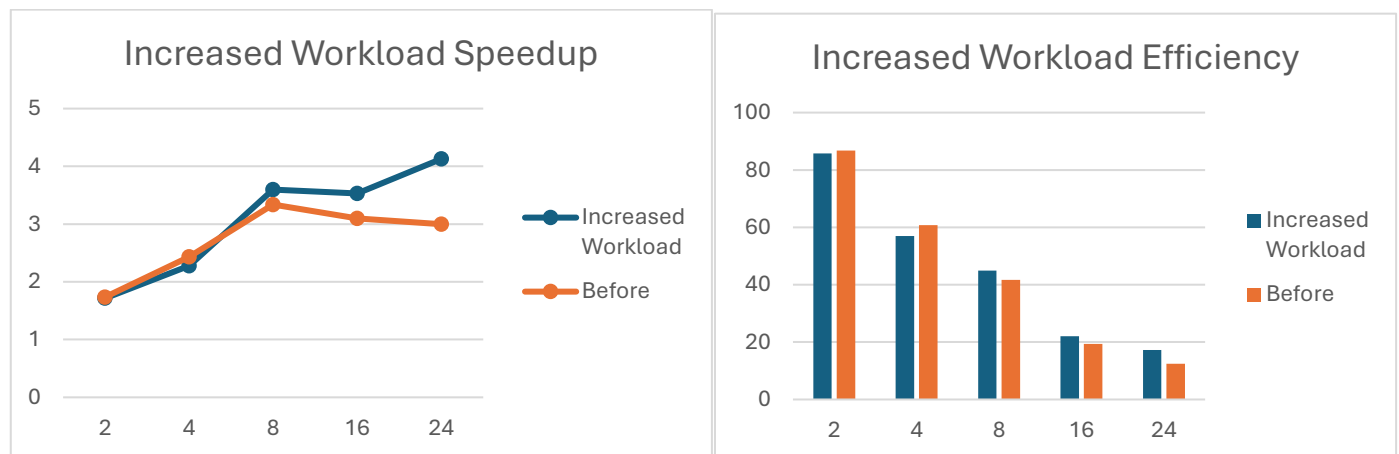


By introducing two threads running in parallel the program runs 1.7x faster. The maximum parallel speedup achieved was just over 3x faster using 8 threads. This is likely due to the program being run on 4 cores and the optimal number of threads each core could manage could be 2 therefore decreasing speedup if more threads are introduced. Increasing the number of threads caused the efficiency to drop, this is due to the speedup not outweighing the cost of managing extra threads.

Table 29 shows an improvement in timings however increasing the workload should demonstrate a clearer improvement from parallelisation.

Increase Workload

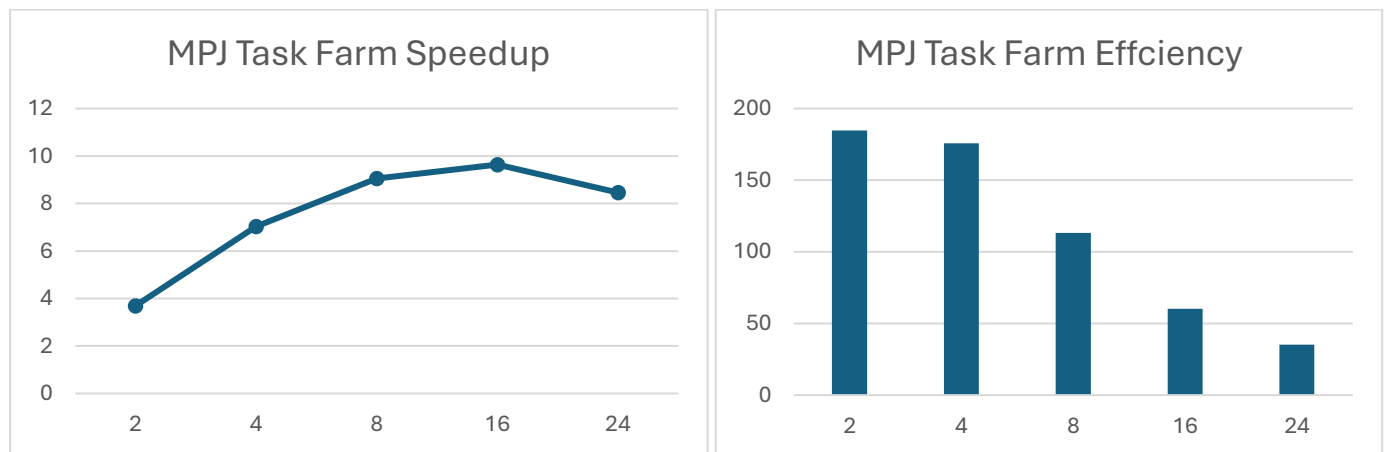
Figure 22: N increased to 10,000,000 Speedup & Efficiency



Multiplying the workload by 100 highlights how increasing the number of threads enhances the programs performance. Initially on two and four threads there is no difference in speed up and the less work is more efficient, however as the threads increase the speed up and efficiency improve upon the previous approach, indicating parallel programs are more suitable for more complex tasks.

MPJ Task Farm

Figure 23: MPJ Task Farm Speedup & Efficiency



Using four worker nodes an MPJ Task farm was created. Increasing the number of processes improved performance as the workload was split into smaller subproblems that could run concurrently. The decrease in speedup at 24 processes could be due to poor task distribution between the workers. It could be argued that the issue is with workers waiting for their next task as there is more tasks than workers. Figure 21's efficiency graph shows relatively similar levels for 2 and 4 processes but then decreases which aligns with the fact there's four worker nodes and would need to wait for a new task if more than four processes. The solution to this would be to increase the number of worker nodes available to balance the workload more effectively.

Parallel Sieve of Eratosthenes

Figure 24: Sequential Vs Sieve of Eratosthenes Best Times

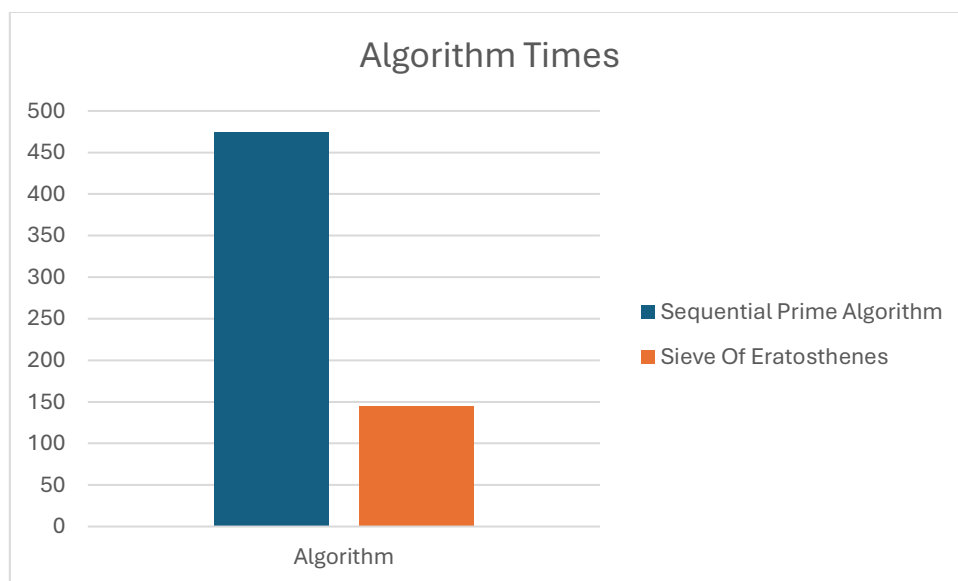
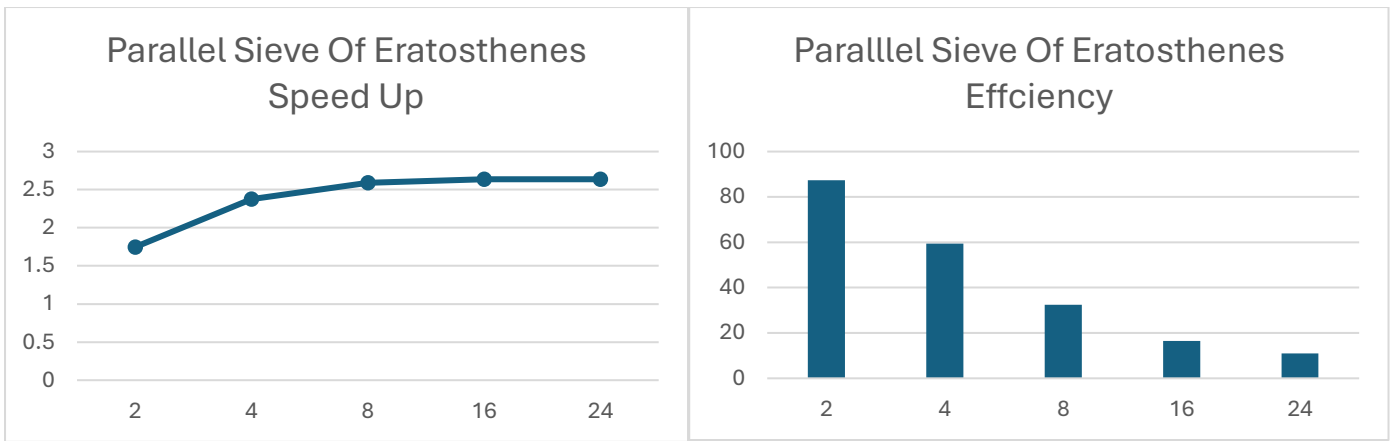


Figure 22 shows that Sieve of Eratosthenes (SofE) algorithm for finding all prime numbers up to N is three times faster than the sequential approach checking each number $2-N$ one at a time, when $N = 10,000,000$.

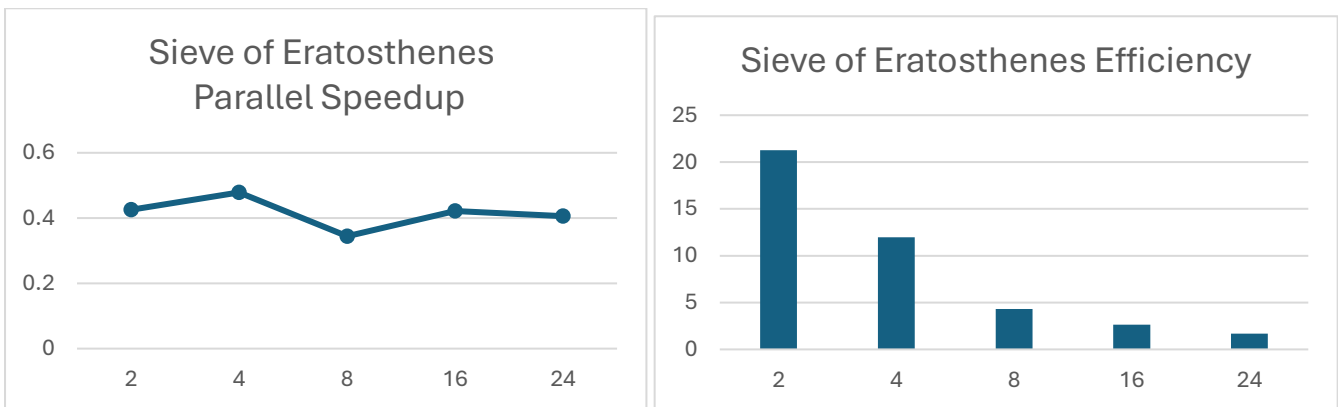
Figure 25: Parallel SofE Speedup & Efficiency



Compared to the sequential approach, the SofE algorithm displays a stagnant growth rate as the number of threads increase. This suggests that this approach is less suitable for parallelising than the original approach.

MPJ Task Farm Sieve of Eratosthenes

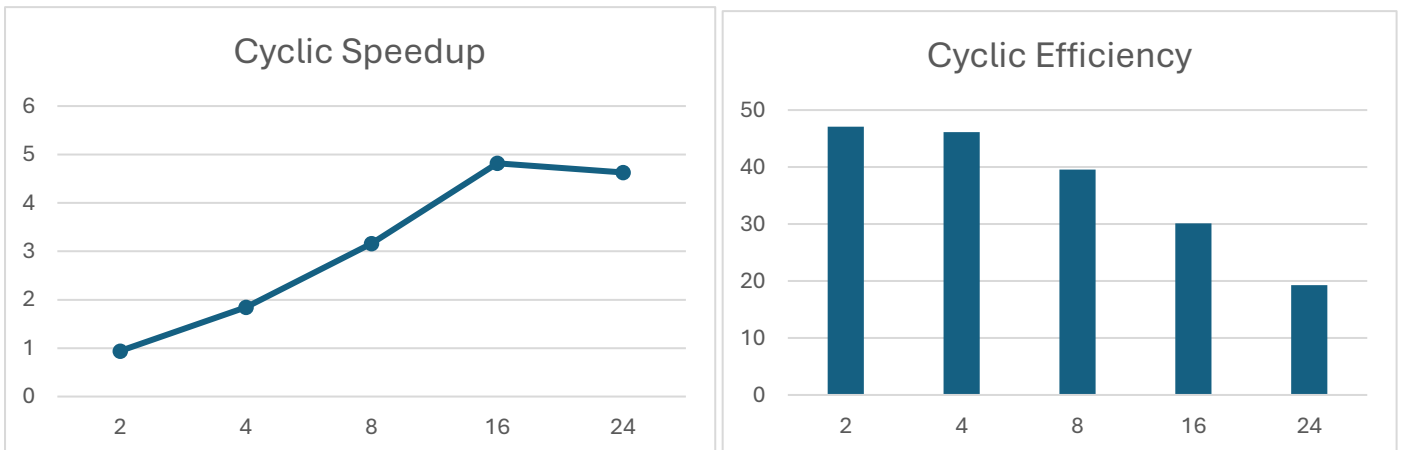
Figure 26: MPJ SofE Speedup & Efficiency



The Sieve of Eratosthenes algorithm has a time complexity of $O(n \log(\log(n)))$. Like the parallel approach, introducing an MPJ task farm showed no increase in speedup as the number of workers increased. In fact this approach exhibited slow down as it ran around 0.4x slower than the sequential approach.

Cyclic Decomposition

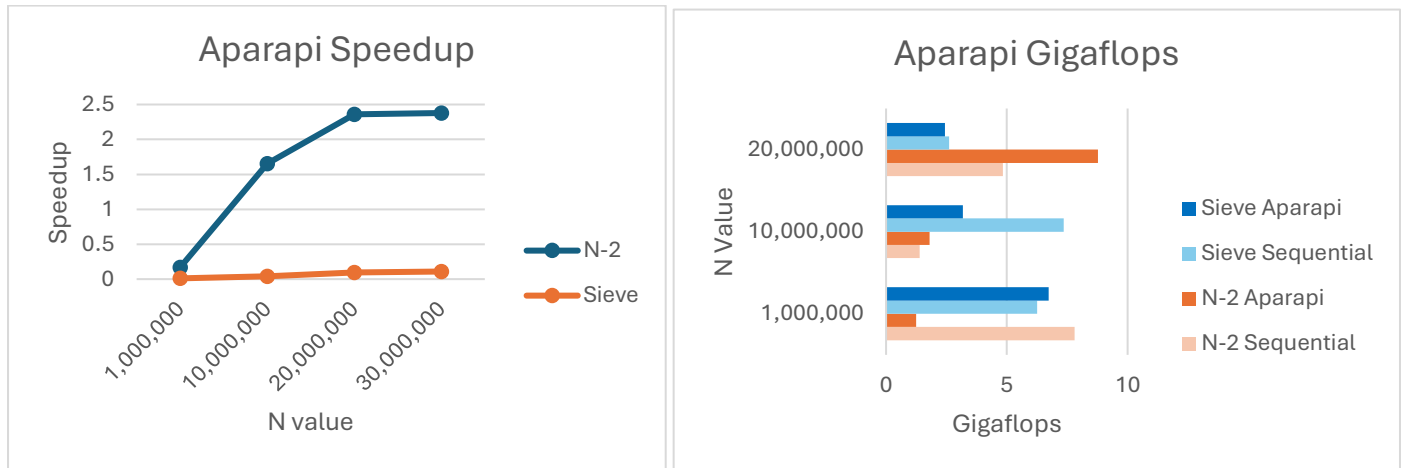
Figure 27: Cyclic Decomposition Speedup & Efficiency



The cyclic decomposition of the 2-N algorithm showed high levels of speed up and maintained higher efficiency as the workload increased.

GPU Aparapi

Figure 28: Aparapi Speedup & Efficiency for Prime Problem



Using Aparapi for GPU programming showed promising speedup for the N-2 algorithm and as the workload increased the aparapi approach provided higher numbers of gigaflops suggesting its better handling larger tasks. On the other hand, the SofE algorithm provided minimal speedup when executing using GPU programming and had lower numbers of gigaflops compared to the sequential approach when the workload was increased.

Results

Figure 29: Parallel Speedup of All CPU Approaches

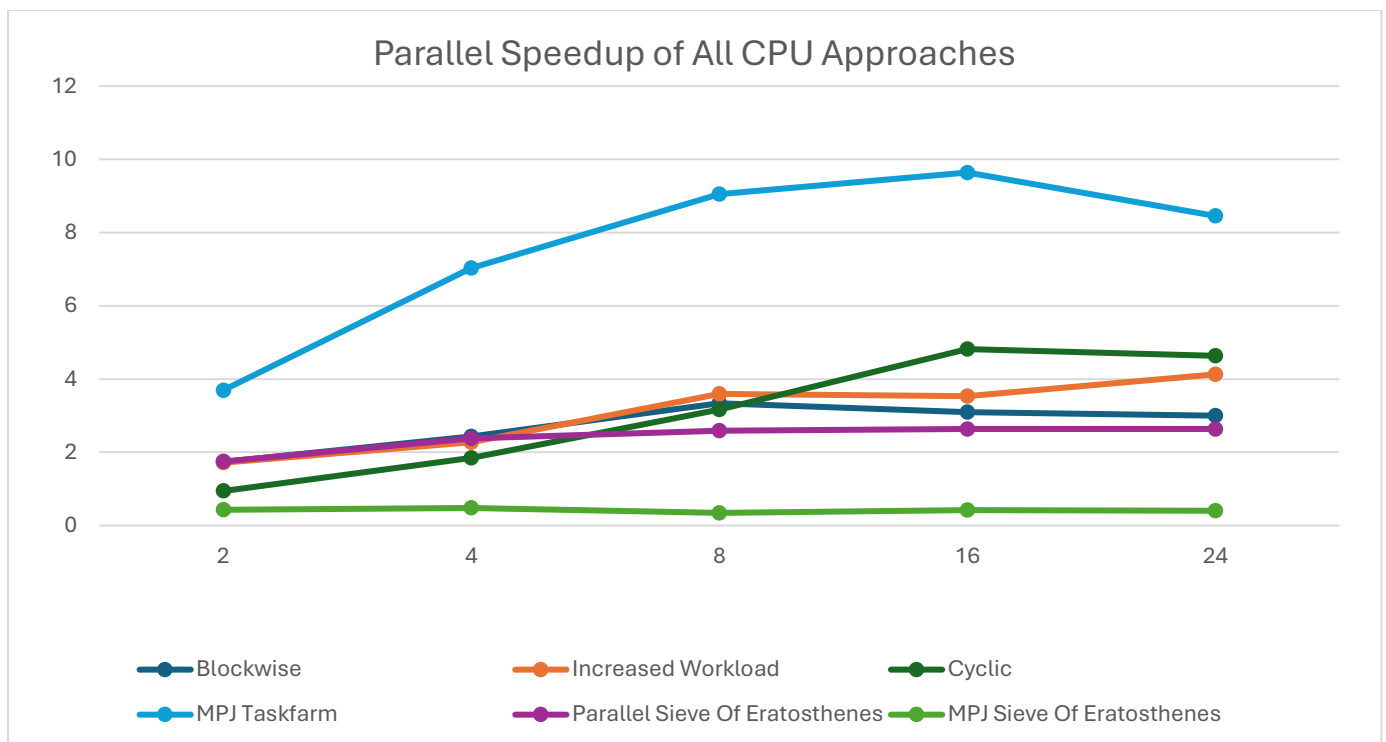
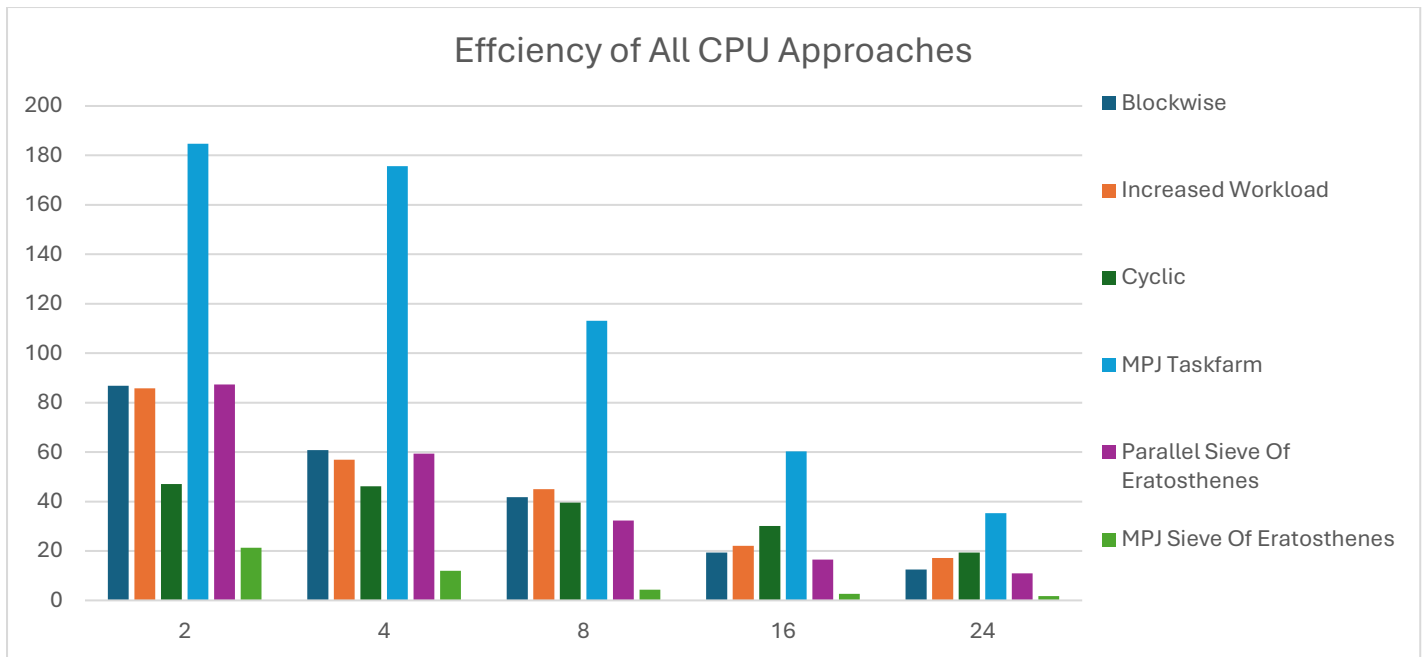
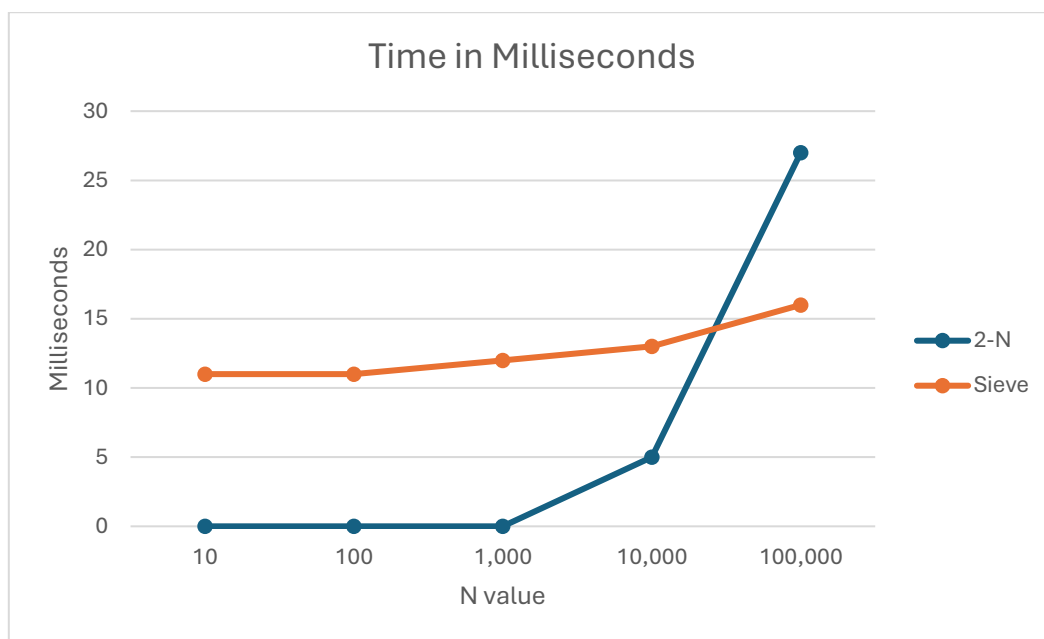


Figure 30: Efficiency of All CPU Approaches



Sequential Vs Sieve of Eratosthenes Times

Figure 31: 2-N vs SofE Times



Conclusion

When N is equal to 10 million, the fastest solution to finding all prime numbers up to N was using the SofE algorithm parallelised on 16 threads. Demonstrated in Figure 31 the larger the workload N, the slower the 2-N algorithm took, whilst the SofE approach remained consistently fast.

Whilst the SofE algorithm was faster, the 2-N algorithm benefited the most from parallelisation, demonstrating a 4x speedup when run on 24 threads. Using MPJ task farms on the 2-N algorithm exhibited the largest parallel speed up and efficiency overall. Despite not being as fast as the parallelized SofE method for larger tasks, the MPJ task farm approach achieved 9x parallel speedup and demonstrated significant efficiency improvements. This can be attributed to the problems embarrassingly parallel nature and how the independent tasks can be distributed to workers with minimal synchronisation overheads.

The second-best performing approach was using cyclic decomposition which executed better as N increased.

The N-2 algorithm also presented to be the better approach to exploit GPU computing using Aparapi due to its speedup and increase in GigaFlops.

Overall, the N-2 algorithm was the best candidate for exploiting parallel programming and architecture as it showed the greatest improvements. Future investigations could include creating an MPI task farm using cyclic decomposition to create an even higher performing solution and observing its execution on GPU architecture.

Conclusion

To be able to successfully analyse program performance, benchmarking timings is key. This allows for performance metrics such as parallel speedup and efficiency to be calculated which can be used to compare approaches and determine/justify their suitability.

When developing parallel solutions, it is important to consider the most effective decomposition technique that provides the most balanced workload. Some problems are not embarrassingly parallel such as simulations and therefore require synchronization to ensure that race conditions do not impact the final result. In this instance barriers are essential to ensure that the parallel approach is precise, however as found in Lab 4 barrier synchronisation can take time which may impact the solutions effectiveness.

Architecture also is a key factor in performance of parallel solutions. Unlike shared memory, distributed memory is better for scalability, however the message passing creates an overhead which may negatively impact speed up. Theoretically, running a program in cluster mode should be more efficient and scalable than multicore as you can exploit the cores of multiple nodes compared to just the one node. As found in Lab 6 this isn't always the case as the overhead of message passing can be detrimental to the implementation. Task farming is great for scalability and resource utilization and works however works best on embarrassingly parallel problems.

The main issue affecting parallel programs is the architecture available. Most of the lab experiments were constrained by the number of CPU's or Nodes available. Increasing these resources should show an even greater improvement in performance to what was achieved in this lab report.